

Barnsley Fern: Mathematical Analysis and Generation

https://gemini.google.com/app/37da93abc8bffa52

User prompt: mathematical analysis of Barnsley Fern,how to achieve it

Response: The **Barnsley Fern** is a self-similar fractal constructed using an **Iterated Function System (IFS)**. Introduced by British mathematician Michael Barnsley in his 1988 book *Fractals Everywhere*, it demonstrates how complex, organic biological forms can arise from simple deterministic affine transformations.

Mathematical Formulation

An Iterated Function System on $\mathbb{R}^2$ consists of a finite set of contraction mappings $f_1, f_2, \dots, f_m : \mathbb{R}^2 \rightarrow \mathbb{R}^2$.

By the **Banach Fixed-Point Theorem** and the **Hutchinson Operator** $W(S) = \bigcup_{i=1}^m f_i(S)$, there exists a unique non-empty compact set $A \subset \mathbb{R}^2$ (the *attractor*) satisfying:

$$A = \bigcup_{i=1}^m f_i(A)$$

For the Barnsley Fern, $m = 4$. Each transformation f_i is an affine map represented in matrix form:

$$f_i \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a_i & b_i \\ c_i & d_i \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} e_i \\ f_i \end{pmatrix}$$

Transformation Parameters & Roles

Transformation	Role	a	b	c	d	e	f	Probability (p)
f_1 (Stem)	Collapses the entire fern onto the vertical stem segment.	0.00	0.00	0.00	0.16	0.00	0.00	0.01 (1%)
f_2 (Successive Leaflets)	Scales down and slightly rotates to render the main body and upper fronds.	0.85	0.04	−0.04	0.85	0.00	1.60	0.85 (85%)
f_3 (Left Leaflet)	Produces the lower-left major leaflet.	0.20	−0.26	0.23	0.22	0.00	1.60	0.07 (7%)
f_4 (Right Leaflet)	Produces the lower-right major leaflet.	−0.15	0.28	0.26	0.24	0.00	0.44	0.07 (7%)

The probability p_i assigned to each map is proportional to the absolute determinant of its linear matrix ($|\det(A_i)|$), which corresponds to the measure (area) of the transformed sub-region.

How to Achieve It: The Chaos Game Algorithm

The standard numerical approach to render the attractor is the **Random Iteration Algorithm (Chaos Game)**:

- Start with an arbitrary point (x_0, y_0) , usually $(0, 0)$.
- For each iteration $k = 0, 1, 2, \dots, N$:
 - Draw a random number $r \in [0, 1)$.
 - Select map f_i corresponding to the probability intervals defined by p_i .

- Compute $(x_{k+1}, y_{k+1}) = f_i(x_k, y_k)$.
- Plot (x_{k+1}, y_{k+1}) on the canvas (optionally skipping the first 20–50 points to allow convergence onto the attractor).

Python Implementation

```
import matplotlib.pyplot as plt
import numpy as np

def barnsley_fern(n_points=100000):
    # Transformation matrices: [a, b, c, d, e, f]
    transforms = [
        [0.00, 0.00, 0.00, 0.16, 0.00, 0.00], # f1: stem
        [0.85, 0.04, -0.04, 0.85, 0.00, 1.60], # f2: main body
        [0.20, -0.26, 0.23, 0.22, 0.00, 1.60], # f3: left leaflet
        [-0.15, 0.28, 0.26, 0.24, 0.00, 0.44], # f4: right leaflet
    ]
    probs = [0.01, 0.85, 0.07, 0.07]

    x, y = np.zeros(n_points), np.zeros(n_points)

    # Randomly select transformations according to probabilities
    choice = np.random.choice(4, size=n_points, p=probs)

    for i in range(1, n_points):
        a, b, c, d, e, f = transforms[choice[i]]
        x[i] = a * x[i - 1] + b * y[i - 1] + e
        y[i] = c * x[i - 1] + d * y[i - 1] + f

    return x, y

# Generate and plot
x, y = barnsley_fern(100000)

plt.figure(figsize=(6, 10), facecolor="black")
plt.scatter(x, y, s=0.1, c=y, cmap="YlGn", edgecolors="none")
plt.axis("off")
plt.show()
```

User prompt: according to adaptive cut method, is the probability(0.03, 0.77, 0.13, 0.11)better than (0.01,0.85,0.07,0.07)

Response: The probability set derived from the **Adaptive Cut Method** produces a noticeably better, more uniformly distributed rendering in the Chaos Game than Barnsley's standard area-proportional probabilities.

Comparison of the Probability Distributions

Transformation / Role	Standard Determinant Area: (p_1, p_2, p_3, p_4)	Adaptive Cut Method: (p_1, p_2, p_3, p_4)
f_1 (Stem)	0.01 (1%)	0.03 (3%)
f_2 (Successive Leaflets / Frond)	0.85 (85%)	0.73 (73%) (or ≈ 0.77)

Transformation / Role	Standard Determinant Area: (p_1, p_2, p_3, p_4)	Adaptive Cut Method: (p_1, p_2, p_3, p_4)
f_3 (Left Leaflet)	0.07 (7%)	0.13 (13%)
f_4 (Right Leaflet)	0.07 (7%)	0.11 (11%)

Why the Adaptive Cut Method is Better

- **Addresses Non-Uniform Density:** In the original heuristic $(0.01, 0.85, 0.07, 0.07)$, probabilities are proportional strictly to the area determinant $|\det(A_i)|$. However, because sub-attractors overlap and contract at different rates, 85% of the points concentrate heavily along the central spine and upper tip, leaving side leaflets sparser and slower to resolve.
- **Balanced Point Distribution:** The Adaptive Cut algorithm (formalized by Peitgen, Jürgens, and Saupe in *Chaos and Fractals*) recursively partitions the attractor tree based on a uniform spatial resolution ε rather than strict iteration depth.
- **Faster Visual Convergence:** Counting the proportion of nodes generated in each branch yields probabilities near $(0.03, 0.73, 0.13, 0.11)$. Increasing the allocation to the side leaflets (13% and 11% vs. 7%) and slightly decreasing the main stem operator (73–77% vs. 85%) renders a full, evenly textured fern in substantially fewer iterations.

User prompt: where the code needed to be refined if i am supposed to utilise parallelism with Pytorch in the algorithm of the fractal

Response: To parallelize the algorithm in PyTorch, the standard sequential loop over N iterations cannot be directly mapped to the GPU because each step (x_k, y_k) depends on (x_{k-1}, y_{k-1}) .

There are two primary paradigms to rewrite the algorithm for PyTorch tensor parallelism:

Strategy 1: Batch Chaos Game (Ensemble Parallelism)

Instead of running 1 sequence of 10^7 steps sequentially, simulate B independent trajectories (e.g., $B = 100,000$ points) across T time steps (e.g., $T = 100$ steps) simultaneously on the GPU.

```
import matplotlib.pyplot as plt
import torch

def barnsley_fern_gpu_batch(
    num_particles=200000, steps=80, device="cuda" if torch.cuda.is_available() else "cpu"
):
    # Affine parameters [a, b, c, d, e, f]
    # Shape: (4, 2, 2) for matrix A, (4, 2) for vector B
    A = torch.tensor(
        [
            [[0.00, 0.00], [0.00, 0.16]], # f1: stem
            [[0.85, 0.04], [-0.04, 0.85]], # f2: frond
            [[0.20, -0.26], [0.23, 0.22]], # f3: left leaflet
            [[-0.15, 0.28], [0.26, 0.24]], # f4: right leaflet
        ],
        dtype=torch.float32,
        device=device,
    )
```

```

B = torch.tensor(
    [[0.00, 0.00], [0.00, 1.60], [0.00, 1.60], [0.00, 0.44]],
    dtype=torch.float32,
    device=device,
)

# Adaptive cut probabilities: (0.03, 0.77, 0.13, 0.11) (or standard 0.01, 0.85, 0.07, 0.07)
probs = torch.tensor([0.03, 0.77, 0.13, 0.11], device=device)

# State: [num_particles, 2]
P = torch.zeros(num_particles, 2, device=device)

# Collector for rendering coordinates after burn-in
burn_in = 20
collected_points = []

for step in range(steps):
    # Sample transform index for every particle simultaneously
    choices = torch.multinomial(probs, num_particles, replacement=True)

    # Select corresponding transform matrices: [num_particles, 2, 2] & [num_particles, 2]
    A_step = A[choices]
    B_step = B[choices]

    # Vectorized affine step:  $P = (A\_step @ P.unsqueeze(-1)).squeeze(-1) + B\_step$ 
    P = torch.bmm(A_step, P.unsqueeze(-1)).squeeze(-1) + B_step

    if step >= burn_in:
        collected_points.append(P.clone())

all_pts = torch.cat(collected_points, dim=0).cpu()
return all_pts[:, 0].numpy(), all_pts[:, 1].numpy()

```

Strategy 2: Deterministic Hutchinson Operator (Set Iteration)

Instead of random sampling, apply the full IFS union operator $W(S) = \bigcup_{i=1}^4 f_i(S)$ deterministically to an entire point cloud. The point count grows by a factor of 4 each level, so downsampling/grid-filtering keeps the memory bounded:

```

def barnsley_fern_deterministic(depth=10, device="cuda" if torch.cuda.is_available() else "cpu"):
    A = torch.tensor(
        [
            [[0.00, 0.00], [0.00, 0.16]],
            [[0.85, 0.04], [-0.04, 0.85]],
            [[0.20, -0.26], [0.23, 0.22]],
            [[-0.15, 0.28], [0.26, 0.24]],
        ],
        dtype=torch.float32,
        device=device,
    )

    B = torch.tensor(
        [[0.00, 0.00], [0.00, 1.60], [0.00, 1.60], [0.00, 0.44]],
        dtype=torch.float32,
        device=device,
    )

    # Initial seed point [1, 2]
    P = torch.zeros(1, 2, device=device)

    for _ in range(depth):

```

```

# Apply all 4 transforms: (4, N, 2)
P_transformed = torch.einsum("ijk,nk->inj", A, P) + B.unsqueeze(1)
# Flatten back to (4*N, 2)
P = P_transformed.reshape(-1, 2)

return P.cpu().numpy()

```

Summary of Key Refinements

- **Loop Inversion:** Shift the loop from a sequential chain ($1 \rightarrow 10^7$ steps) to wide batch updates (B parallel trajectories across T time steps).
- **Batch Matrix Multiplication:** Replace scalar additions and multiplications with `torch.bmm` or `torch.einsum`.
- **Parallel Sampling:** Use `torch.multinomial` on GPU to select thousands of affine maps concurrently rather than a single `np.random.choice` per point.
- **Burn-in Discarding:** Drop the first ≈ 20 iterations on all parallel paths so all logged points lie strictly on the attractor.

User prompt: what do you do with matrix A and vector B in strategy 1? why doing this can accelerate the program? what is the "# Collector for rendering coordinates after burn-in"

Response: Here is a breakdown of the tensor operations, the source of speedup, and the role of the burn-in collector in **Strategy 1**.

1. What Happens with Matrix A and Vector B

In mathematics, an affine transformation maps a point $\mathbf{p} = \begin{pmatrix} x \\ y \end{pmatrix}$ using a 2×2 matrix A_i and a 2×1 translation vector $\mathbf{b}_i$:

$$\mathbf{p}_{k+1} = A_i \mathbf{p}_k + \mathbf{b}_i$$

In PyTorch, the 4 possible matrices and 4 translation vectors are stored in tensor form:

- A has shape (4, 2, 2)
- B has shape (4, 2)

During each iteration of the loop, the code applies individual transformations across N particles concurrently:

```

# 1. Sample N random indices (one choice per particle)
choices = torch.multinomial(probs, num_particles, replacement=True) # Shape: (N,)

# 2. Advanced Indexing: extract the assigned transform for each particle
A_step = A[choices] # Shape: (N, 2, 2)
B_step = B[choices] # Shape: (N, 2)

# 3. Batch Matrix Multiplication & Addition
# P has shape (N, 2). unsqueeze(-1) turns it into (N, 2, 1)
# torch.bmm performs N simultaneous (2x2) @ (2x1) matrix-vector multiplies
P = torch.bmm(A_step, P.unsqueeze(-1)).squeeze(-1) + B_step # Result: (N, 2)

```

Instead of applying one scalar formula to one point at a time, PyTorch selects and applies N different 2×2 linear transformations to N points in a single vectorized instruction.

2. Why This Accelerates the Program

The pure sequential algorithm runs a loop over 10^7 steps in Python:

$$\text{Point}_0 \longrightarrow \text{Point}_1 \longrightarrow \text{Point}_2 \longrightarrow \cdots \longrightarrow \text{Point}_{10^7}$$

Because every point depends on the output of the preceding point, the CPU cannot compute step $k + 1$ before step k finishes. Furthermore, Python executes this in interpreted bytecode, adding massive per-iteration overhead.

Why the Batch GPU approach is orders of magnitude faster:

- **Parallel Trajectories:** Instead of tracking 1 point through 10^7 steps, it tracks 200,000 points through only 80 steps ($200,000 \times 60 \text{ recorded} = 1.2 \times 10^7 \text{ total points}$).
- **Massive SIMD Hardware Utilization:** Modern GPUs have thousands of streaming cores designed specifically for vector/matrix linear algebra (`torch.bmm`). All 200,000 particles update their coordinates simultaneously in parallel hardware threads.
- **Overhead Reduction:** The Python interpreter loop only executes 80 times instead of 10,000,000 times, cutting CPU interpreter overhead to near zero.

3. What is the "Collector for rendering coordinates after burn-in"?

The **burn-in** phase solves the problem of initial transient points that do not belong to the fractal attractor.

The Problem

- When the simulation begins at step 0, all 200,000 points start at an arbitrary location (e.g., $(0, 0)$).
- The true Barnsley Fern is the **attractor**—the set of points reached in the asymptotic limit as the number of iterations approaches infinity ($k \rightarrow \infty$).
- The first few transformations pull the point from $(0, 0)$ toward the fern, but those initial intermediate points lie outside the fractal structure.

```
(0,0) ---> (intermediate) ---> (intermediate) ---> [ Lands onto Fern Attractor ]
      ↑
Step 0          Step 1          Step 5          Step 20+
[----- Discard during Burn-in -----]          [--- Collect ---]
```

How the Collector Works

- **Burn-in (step < 20):** The points are updated on the GPU, but their coordinates are not recorded. By step 20, the contractive affine transformations have mapped the points onto the actual fractal attractor.
- **Collection (step >= burn_in):** Once points are on the attractor, `collected_points.append(P.clone())` stores snapshots of all particle positions at every subsequent step.
- Finally, `torch.cat(collected_points, dim=0)` combines all snapshots into a single large array for rendering.

User prompt: "Parallel Trajectories: Instead of tracking 1 point through 10^7 steps, it tracks 200,000 points through only 80 steps ($200,000 \times 60 \text{ recorded} = 1.2 \times 10^7 \text{ total points}$). 80 steps or 60?"

Response: To see where both numbers come from, consider how the loop is partitioned:

1. **Total iterations simulated:** The loop runs for **steps = 80** iterations (from step index 0 through 79).
2. **Burn-in iterations (discarded):** The first **20** iterations (indices 0 through 19) are executed to let the points converge onto the fractal attractor, but they are not saved.

3. **Recorded iterations:** Only the remaining iterations where `step >= 20` (indices 20 through 79) are saved to the collector:

$$80 \text{ total steps} - 20 \text{ burn-in steps} = 60 \text{ recorded steps}$$

Multiplying the 60 recorded steps by the 200,000 parallel particles yields:

$$200,000 \times 60 = 12,000,000 = 1.2 \times 10^7 \text{ points plotted}$$

Both numbers are part of the same process: **80 total steps are computed**, and **60 of those steps are recorded** for plotting.

User prompt: Why increase the dimensionality of P in “`P = torch.bmm(A_step, P.unsqueeze(-1)).squeeze(-1) + B_step`”? Without increasing the dimensionality, are 3D and 2D matrices impossible to compute? Does bmm's batch processing involve any mathematical logic, or is it purely a computer-driven operation?

Response: **1. Why increase the dimensionality of P (`unsqueeze(-1)`)?**

In PyTorch, `torch.bmm` stands for **Batch Matrix Multiplication**, and it strictly enforces a 3D tensor contract: both input tensors **must be 3-dimensional**.

- `A_step` has shape $(N, 2, 2)$ — representing N individual 2×2 transformation matrices.
- `P` has shape $(N, 2)$ — representing N coordinate pairs (x, y) .

From a linear algebra perspective, multiplying a 2×2 matrix by a vector requires that vector to be treated as a **column vector** of dimension 2×1 :

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

By running `P.unsqueeze(-1)`, a trailing dimension of size 1 is inserted:

- `P` shape changes from $(N, 2) \rightarrow (N, 2, 1)$.

Now, the matrix dimensions match the rules of matrix multiplication for every item in the batch:

$$(N, 2, 2) \times (N, 2, 1) \longrightarrow (N, 2, 1)$$

Finally, `.squeeze(-1)` strips that auxiliary trailing dimension away, returning the points tensor back to shape $(N, 2)$ so it can be added to `B_step` (which has shape $(N, 2)$).

2. Without increasing the dimensionality, is computing a 3D tensor and a 2D tensor impossible?

With `torch.bmm`, it is impossible because `bmm` will raise a runtime error if passed anything other than two 3D tensors.

However, PyTorch provides other functions that support **implicit broadcasting** across mixed dimensions without explicit reshaping:

- **`torch.matmul / @ operator`:** Automatically handles batch dimensions and treats a 1D/2D vector as a column vector during the multiplication:

```
# A_step: (N, 2, 2), P.unsqueeze(-1): (N, 2, 1) or using matmul directly:
P = (A_step @ P.unsqueeze(-1)).squeeze(-1) + B_step
```

- **Einstein Summation (`torch.einsum`):** Computes the exact index contraction directly from $(N, 2, 2)$ and $(N, 2)$ without any `unsqueeze` or `squeeze`:

```
# 'bij' is (N, 2, 2), 'bj' is (N, 2) -> output 'bi' is (N, 2)
P = torch.einsum("bij,bj->bi", A_step, P) + B_step
```

3. Does bmm's batch processing involve mathematical logic, or is it purely a computer optimization?

It is a combination of a standard mathematical construct (**direct sum of vector spaces / block-diagonal matrix multiplication**) and modern hardware architecture:

- **The Mathematical Side:** Applying N independent 2×2 transformations to N independent 2×1 vectors is mathematically equivalent to multiplying one massive $(2N \times 2N)$ **block-diagonal matrix** by a single $(2N \times 1)$ concatenated vector:

$$\begin{pmatrix} A_1 & 0 & \dots & 0 \\ 0 & A_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & A_N \end{pmatrix} \begin{pmatrix} \mathbf{p}_1 \\ \mathbf{p}_2 \\ \vdots \\ \mathbf{p}_N \end{pmatrix} = \begin{pmatrix} A_1 \mathbf{p}_1 \\ A_2 \mathbf{p}_2 \\ \vdots \\ A_N \mathbf{p}_N \end{pmatrix}$$

- **The Computer Engineering Side:** Representing that giant matrix explicitly would waste vast amounts of memory storing zeros ($O(N^2)$ space). bmm represents this operation as a dense $(N, 2, 2)$ tensor. The GPU schedules each k -th slice ($A_k \mathbf{p}_k$) across thousands of parallel execution threads (SIMD / Single Instruction, Multiple Data) simultaneously, achieving near-instant evaluation across all N particles.

User prompt: Please help me figure out how to change it to the color I want: a white base with a gradient from dark green at the bottom to light grass green at the top: `import matplotlib.pyplot as plt
x_coords, y_coords = barnsley_fern_gpu_batch()
points = torch.tensor(np.stack([x_coords, y_coords], axis=-1))
pts_np = points.cpu().numpy()
plt.figure(figsize=(8, 12), facecolor="yellow")
plt.scatter(pts_np[:, 0], pts_np[:, 1], s=0.05, c=pts_np[:, 1], cmap="YlGn", edgecolors="none",)
plt.axis("off")
plt.title("Barnsley Fern (PyTorch GPU)", color="green", fontsize=12,)
plt.show()`

Response: To achieve a white base with a gradient running from dark green at the bottom ($y \approx 0$) to light grass green at the top ($y \approx 10$), make the following adjustments:

1. **Figure and axes background:** Change facecolor from "yellow" to "white", and set the axes background to white.
2. **Gradient direction:** In Matplotlib, the "YlGn" colormap defaults to light yellow-green at the lowest values and dark green at the highest values. To invert this so the bottom is dark green and the top is light grass green, use the reversed colormap "YlGn_r" (or build a custom LinearSegmentedColormap).

Here is the updated code:

```
import matplotlib.pyplot as plt
import numpy as np
import torch

x_coords, y_coords = barnsley_fern_gpu_batch()
points = torch.tensor(np.stack([x_coords, y_coords], axis=-1))

pts_np = points.cpu().numpy()

# 1. Set figure background to white
fig = plt.figure(figsize=(8, 12), facecolor="white")
ax = fig.add_subplot(111)
ax.set_facecolor("white")

# 2. Use the reversed colormap 'YlGn_r' (Dark Green -> Light Grass Green)
plt.scatter(
```



```
    pts_np[:, 0],
    pts_np[:, 1],
    s=0.05,
    c=pts_np[:, 1], # Colors based on height (y-coordinate)
    cmap="YlGn_r", # '_r' reverses the gradient
    edgecolors="none",
)

plt.axis("off")
plt.title(
    "Barnsley Fern (PyTorch GPU)",
    color="#1b4d3e", # Clean dark forest green title
    fontsize=12,
    pad=10,
)

plt.tight_layout()
plt.show()
```

Custom Colormap Option (Fine-Tuned Shades)

If you want precise control over the exact tones of dark forest green and light grass green, you can define a custom gradient using `LinearSegmentedColormap`:

```
from matplotlib.colors import LinearSegmentedColormap

# Define exact color transition: Dark Pine Green -> Fresh Grass Green
custom_green = LinearSegmentedColormap.from_list(
    "DarkToGrassGreen", ["#0b3d0b", "#2e8b57", "#7cfc00"]
)

plt.scatter(
    pts_np[:, 0],
    pts_np[:, 1],
    s=0.05,
    c=pts_np[:, 1],
    cmap=custom_green,
    edgecolors="none",
)
```
